

# Code and Data Availability Statement

Dataset-Free Inverse Design of Electrostatic MEMS  
via the Pull-In Bifurcation

Tushar Dudeja, *Student Member, IEEE*, and Nawaz Shafi, *Member, IEEE*  
Vellore Institute of Technology, India  
Track 2: AI for MEMS

---

## 1. Repository

All code, figures, result arrays and trained weights are public:

<https://github.com/thetushardudeja1/mems-differentiable-pullin>

public · MIT licensed · no commercial software required

### 1.1 Run it without installing anything

Two notebooks open directly in Google Colab. Nothing needs to be cloned or installed — each notebook clones the repository in its first cell. A free Google account and the free CPU runtime are sufficient; no GPU, no licence and no dataset are involved. Outputs are stored in both files, so each can also be read on GitHub without being run.

**1. Validation and results** ( $\sim 4$  min). Reproduces the validation suite against all eight published sources, the exact-gradient identity check, the optimal-exponent reversal in both boundary conditions, and the  $22.86 \rightarrow 14.04$  V inverse-design descent, live.

[https://colab.research.google.com/github/thetushardudeja1/mems-differentiable-pullin/blob/main/notebooks/MEMS\\_differentiable\\_pullin.ipynb](https://colab.research.google.com/github/thetushardudeja1/mems-differentiable-pullin/blob/main/notebooks/MEMS_differentiable_pullin.ipynb)

**2. How the method works** ( $\sim 3$  min). Four electrodes designed from scratch, the exact shape sensitivity  $\partial V_{PI}/\partial D(\xi)$ , design straight from a specification in 0.04 ms, and adaptive control over 512 devices.

[https://colab.research.google.com/github/thetushardudeja1/mems-differentiable-pullin/blob/main/notebooks/method\\_deep\\_dive.ipynb](https://colab.research.google.com/github/thetushardudeja1/mems-differentiable-pullin/blob/main/notebooks/method_deep_dive.ipynb)

## 2. Code

The repository is self-contained: a differentiable MEMS pull-in solver, its validation suite, the inverse-design and amortized-design pipelines, the reinforcement-learning environment and policy, the surrogate baselines we benchmarked ourselves against, and the full figure and table generation chain.

| Component                                            | Files                                                                               |
|------------------------------------------------------|-------------------------------------------------------------------------------------|
| Notebook — validation and results                    | notebooks/MEMS_differentiable_pullin.ipynb                                          |
| Notebook — method deep dive                          | notebooks/method_deep_dive.ipynb                                                    |
| Differentiable solver (fold system, exact gradients) | sim/beam.py                                                                         |
| Lumped dynamic model (GPU-batched)                   | sim/pullin.py                                                                       |
| Validation suite                                     | test_fold.py, validate_beam.py, validate_nazemi.py, validate_mtest.py, model2dof.py |
| Inverse design                                       | sim/inverse_design.py                                                               |
| Amortized design network                             | amortized_hard.py, amortized_design.py, amortized_vs_converged.py                   |
| RL environment and policy                            | env2dof.py, train_rl2dof.py, analyze_rl2dof.py                                      |
| Surrogate baselines (MLP, FNO)                       | surrogate_fair.py, fno_surrogate.py, surrogate_vs_direct.py                         |
| Sensitivity map                                      | sensitivity_map.py, make_fig_sensitivity.py                                         |
| Benchmarks                                           | sim/bench.py, bench_one.py                                                          |
| Figures and tables                                   | gen_figdata.py, gen_figdata2.py, make_figures.py, make_tables.py                    |

### 3. Data

**This project uses no training dataset.** That is the central methodological point: because the physics is differentiable, every gradient comes from a live solve rather than from an archive of simulations. The amortized design network was trained with **6,000 live solves and 0 stored samples**. The surrogate baselines we built for comparison are the only components that consume sampled data, and those samples are generated on demand by the included scripts.

**Validation targets** are numerical values transcribed from published papers — geometry, material properties and measured voltages. Each value is cited at the point of use in the source file, and `papers/README.md` records what every reference is used for. **Third-party PDFs are not redistributed**; they are excluded via `.gitignore`, and the index gives full citations and DOIs so any reader can obtain them from the publisher.

**Result arrays are committed.** The arrays behind every figure (`sim/figdata_*.npz`, `sim/*.npy`, 84kB in total) are in the repository, so the notebooks can display the three long experiments after a fresh clone without re-running them. They are fully regenerable with `gen_figdata.py`.

**Trained weights are committed too.** `sim/amort_hard_theta.pkl` (42 kB, the amortized design network) and `sim/rl_policy.pkl` (38 kB, the adaptive control policy) are both small enough to publish, so the released results can be loaded and re-evaluated directly rather than only described. The training scripts regenerate them from scratch.

## 4. Reproducing Everything

```
conda env create -f environment.yml
conda activate mems
cd sim

# every fast experiment, live, ~4 min on a CPU
jupyter lab ../notebooks/MEMS_differentiable_pullin.ipynb

# validation: each prints PASS/FAIL against published values
python test_fold.py
python validate_beam.py
python model2dof.py
python validate_nazemi.py
python validate_mtest.py

# where the leverage is: autodiff vs. finite differences
python sensitivity_map.py

# regenerate all figure data, then the figures and tables
python gen_figdata.py && python gen_figdata2.py
python make_figures.py && python make_tables.py
```

Runtime is a few minutes per validation script and roughly 30 minutes for the full figure-data regeneration, on a laptop CPU. The long experiments — the amortized network (26 min), three RL seeds (8 min) and the surrogate baselines (10 min) — are listed with their commands in the repository README.

**No commercial licence (COMSOL, ANSYS, MATLAB) is required at any point.** This is a deliberate design goal, since licence cost is itself a barrier to reproducing MEMS design work.

## 5. Environment

Python 3.11, JAX 0.10.2, NumPy and Matplotlib, pinned in `environment.yml`. Developed on WSL2 (Ubuntu) with an RTX 4060.

**Hardware note.** The static beam solver requires float64: the fourth-order stencil is cancellation-limited and fp32 does not converge, with the fold residual stalling at 2.0 instead of  $10^{-8}$ . Consumer GPUs run fp64 at roughly 1/64 of their fp32 rate, so the solver is **CPU-bound by design and no result is GPU-dependent**. The GPU is used only for the fp32 dynamics and RL training, where it gives about  $8\times$ .

## 6. Determinism

All stochastic components use explicit, seeded JAX PRNG keys, so runs are reproducible. Reinforcement-learning results are reported across three independent seeds with standard deviations, on a fixed held-out set of 512 devices.

## 7. Licence

Everything in the repository — code, figures and documents — is released under the **MIT Licence**: use, modification and redistribution are permitted, including commercial use, with attribution. See `LICENSE` in the repository root. Third-party quotations and transcribed validation values remain the property of their respective copyright holders, as recorded in `NOTICE`.